

프로그래밍 언어 명세와 이모저모 (3/3)

앞선 글에서 우리는 프로그래밍 언어가 무엇인지, 프로그래밍 언어의 명세는 무엇인지, 프로그래밍 언어의 명세의 기술법에는 어떤 것이 있는지, 이상적인 명세는 무엇을 기술하여야 하는지 논의하였다. 이번 글에서는 앞서 언급된 여러 문제를 해결하는 방법 중 하나인, 기계화 (Mechanization)에 대하여 더 자세히 다룬다.

기계화는 특정 대상(이번 글에서는 명세)을 기계가 다룰 수 있는 형태로 만드는 일이다. 기계가 명세의 내용을 해석하고 그에 따라 연산하기 위해서는, 명세의 구조와 의미가 기계가 처리할 수 있는 명시적인 표현으로 주어져야 한다. 따라서 명세의 기계화는 일반적으로 명세를 특정한 형식의 표현으로 기술하는 형식화를 포함한다. 또한 명세가 기계적으로 표현되면, 그 구조와 정의에 대해 문법적 오류나 일부 정의되지 않은 경우와 같은 기본적인 문제를 자동으로 확인하는 것도 가능해진다.

이번 글에서 주목할 장점은, 언어와 관련된 여러 산출물을 기계적으로 생성 가능하다는 점이다. 언어와 관련된 산출물의 종류는 다양하다. 언어와 관련하여 수행할 수 있는 특정 능력이 될 수도 있고, 도구가 될 수도 있으며, 예제가 될 수도 있다. 예를 들어 ITP를 이용하면 명세에 대한 메타이론적 논증을 수행할 수 있고, Maude와 같은 도구를 이용하면 LTL model checking을 수행할 수 있다. 또한 SpecTec과 같은 도구에서는 명세의 실행과 분석, 예제 생성 등의 기능을 제공한다.

이처럼 명세를 여러 작업과 산출물의 공통된 기반으로 사용할 수 있다는 점은 프로그래밍 언

어의 개발에서도 중요하다. 자동화된 생성이 제공되지 않더라도, 명세를 참고하여 실행기, 분석기 등의 산출물을 각각 직접 구현할 수는 있다. 그러나 이 경우 명세와 각각의 산출물이 서로 독립적으로 작성되고 변경될 수 있기 때문에, 두 대상의 의미가 일치하는지를 지속적으로 확인해야 한다. 반면 명세를 기반으로 산출물을 기계적으로 생성하거나 처리할 수 있다면, 산출물을 새로 구현하거나 수정하는 데 필요한 반복적인 노력을 줄일 수 있다. 더 나아가 생성기의 정확성이 검증되어 있다면, 생성된 산출물이 명세의 정의를 올바르게 반영한다는 것을 보장하는 데에도 활용할 수 있다. 이러한 점은 프로그래밍 언어의 프로토타이핑과 유지보수에 중요한 장점이 된다.

두 번째 글에서 논의한 바와 같이, 프로그래밍 언어는 컴퓨터가 특정 계산을 수행하도록 하기 위한 언어이다. 따라서 여러 산출물 중에서도 프로그램을 실제로 실행하여 그 계산을 수행할 수 있도록 하는 산출물은 특히 중요하다.

이에 이번 서베이에서는 프로그래밍 언어의 명세를 기계화하는 도구 중에서도, 명세가 정의하는 계산을 직접 수행할 수 있는 실행 능력을 자동으로 제공하는 도구에 초점을 맞추어 조사한다. 경우에 따라 이는 표기법 자체가 가지는 능력일 수도 있고, 도구가 제공하는 능력일 수도 있다. 여기서 실행 능력은 명세를 작성하는 표기법 자체가 제공할 수도 있고, 해당 표기법으로 작성된 명세를 처리하는 도구가 제공할 수도 있다. 우선 실제로 사용 가능한 도구를 중심으로 조사하며, 각 도구에서 요구하는 표기법이 무엇인지, 그리고 그 표기법으로 기술된 명세를 어떤 방식으로 실행 가능하게 만드는지를 정리한다.